UNIVERSIDADE FEDERAL DO CEARÁ

Relatório Final Modelo

Disciplina: Qualidade de Software
Equipe: Francisco Henrique Nunes da Silva (566487) e Tobias Alves Couras (565022)
Projeto: Projeto Jarvis Original
Fork: Projeto Jarvis Fork
Pull Request: Link do PR
Commit SHA: 94c54491eba58d63c44f044695d12c61224e45a0

1. Apresentação do projeto

O Jarvis é um assistente pessoal de código aberto, desenvolvido em Python, que utiliza inteligência artificial para interpretar comandos, interagir com o usuário e executar diferentes tarefas. O sistema oferece funcionalidades como interação por voz e texto, gerenciamento de memória, integração com ferramentas externas e automação de atividades. Seu público-alvo são usuários que desejam um assistente inteligente e desenvolvedores interessados em estudar, personalizar ou ampliar suas funcionalidades.

Os testes do projeto estão organizados principalmente no diretório tests/, utilizando o framework pytest. Eles são distribuídos em arquivos de acordo com as funcionalidades do sistema, como gerenciamento de conversas, reprodução de áudio, integração com ferramentas MCP e controle de estados. O projeto possui testes unitários e testes de integração, incluindo testes marcados como e2e (ponta a ponta). Essa organização facilita a identificação de falhas, a manutenção do código e a aplicação de refatorações para remover Test Smells, melhorando a clareza, a confiabilidade e a eficiência dos testes.

2. Estado inicial do projeto

2.1 Test smells detectados antes da refatoração (TEMPY)

Test smell	Ocorrências detectadas
Conditional Test Logic	744
Redundant Print	418
Exception Handling	147

Test smell	Ocorrências detectadas
Verbose Test	140
Unknown Test	87
Sleepy Test	85
Programming Paradigms Blend	79
Non-Functional Statement	22
Undefined Test	18
Total	1740

2.2 Resultados dos testes e cobertura antes da refatoração (Pytest)

Métrica	Valor
Testes aprovados (Passed)	3402
Testes com falha (Failed)	8
Erros (Errors)	0
Ignorados (Skipped)	12
Cobertura total (%)	70,52

2.3 Métricas do código de teste antes da refatoração (Radon)

Métrica	Valor
SLOC total dos testes (linha TOTAL com origem = teste)	36.438
Complexidade ciclomática média das funções de teste	2,96
Índice de manutenibilidade médio dos arquivos de teste	46,70

3. Refatorações realizadas

3.1 Seleção das ocorrências

Foram escolhidos quatro tipos de Test Smells identificados com o auxílio do CATSPY: Conditional Test Logic, Sleepy Test, Unknown Test e Verbose Test. A seleção considerou os problemas encontrados no código e seus possíveis impactos na qualidade dos testes.

O Conditional Test Logic foi escolhido porque o uso de estruturas condicionais dentro dos testes pode dificultar a compreensão e permitir que determinados cenários não sejam verificados corretamente. O Sleepy Test foi selecionado por utilizar esperas fixas, que podem tornar os testes mais lentos e gerar resultados inconsistentes. Já o Unknown Test foi escolhido por apresentar testes sem verificações explícitas, dificultando a identificação de falhas. Por fim, o Verbose Test foi selecionado por conter testes com código excessivo e informações desnecessárias, prejudicando a legibilidade e a manutenção.

Esses quatro tipos foram considerados relevantes por afetarem diferentes aspectos da qualidade dos testes, como confiabilidade, desempenho, clareza e facilidade de manutenção. As refatorações buscaram reduzir esses problemas sem alterar o comportamento esperado do sistema.

3.2 Lista das refatorações

ID	Test smell	Arquivo::teste	Commit (hash curto)	Testes passaram?	Removido pelo TEMPY?	Introduziu novo smell?
R01	Conditional Test Logic	test_mcp_e2e.py	94149ba	sim	sim	não
R02	Conditional Test Logic	test_mcp_e2e.py	9663a21	sim	sim	não
R03	Conditional Test Logic	test_mcp_e2e.py	dc71d0c	sim	sim	não
R04	Conditional Test Logic	test_mcp_e2e.py	0ac01fb	sim	sim	não
R05	Conditional Test Logic	test_mcp_e2e.py	bcc242e	sim	sim	não
R06	Conditional Test Logic	test_mcp_e2e.py	06bd431	sim	sim	não
R07	Conditional Test Logic	test_mcp_e2e.py	60ea9d6	sim	sim	não
R08	Conditional Test Logic	test_mcp_e2e.py	f60e97c	sim	sim	não
R09	Conditional Test Logic	test_tool_selection.py	08392d9	sim	sim	não

ID	Test smell	Arquivo::test e	Commit (hash curto)	Testes passaram?	Removido pelo TEMPY?	Introduziu novo smell?
R10	Conditional Test Logic	test_local_files_path_arguments.py	3f71ddd	sim	sim	não
R11	Conditional Test Logic	test_tools.py	144f828	sim	sim	não
R12	Sleepy Test	test_chat_submission.py	bc8dd2a	sim	sim	não
R13	Sleepy Test	test_chat_submission.py	ba28211	sim	sim	não
R14	Sleepy Test	test_chat_submission.py	d46cee0	sim	sim	não
R15	Sleepy Test	test_chat_submission.py	2c3f910	sim	sim	não
R16	Sleepy Test	test_state_manager.py	8ac1090	sim	sim	não
R17	Sleepy Test	test_state_manager.py	f06868a	sim	sim	não
R18	Sleepy Test	test_state_manager.py	e879e68	sim	sim	não
R19	Sleepy Test	test_state_manager.py	2b3a139	sim	sim	não
R20	Sleepy Test	test_state_manager.py	eae9130	sim	sim	não
R21	Sleepy Test	test_state_manager.py	dc8f813	sim	sim	não
R22	Sleepy Test	test_state_manager.py	6d2f86f	sim	sim	não
R23	Sleepy Test	test_state_manager.py	ec63ac8	sim	sim	não
R24	Sleepy Test	test_state_manager.py	b695c0f	sim	sim	não
R25	Unknown Test	test_chat_submission.py	f903d60	sim	sim	não
R26	Unknown Test	test_chat_submission.p	3674982	sim	sim	não

ID	Test smell	Arquivo::test e	Commit (hash curto)	Testes passaram?	Removido pelo TEMPY?	Introduziu novo smell?
		y				
R27	Unknown Test	test_graph_mutation_listener.py	218d93b	sim	sim	não
R28	Unknown Test	test_tune_player.py	bc6f9cd	sim	sim	não
R29	Unknown Test	test_desktop_app.py	328d2e4	sim	sim	não
R30	Unknown Test	test_piper_tts.py	adc1b88	sim	sim	não
R31	Unknown Test	test_piper_tts.py	87633ba	sim	sim	não
R32	Unknown Test	test_piper_tts.py	34ee927	sim	sim	não
R33	Unknown Test	test_dictation.py	84817aa	sim	sim	não
R34	Unknown Test	test_dictation.py	95e8cc1	sim	sim	não
R35	Unknown Test	test_chat_window.py	eb831e7	sim	sim	não
R36	Verbose Test	test_mcp_e2e.py	055134f	sim	sim	não
R37	Verbose Test	test_mcp_e2e.py	d1e7dab	sim	sim	não
R38	Verbose Test	test_dialogue_memory.py	3abe309	sim	sim	não
R39	Verbose Test	test_dictation_history.py	139ab48	sim	sim	não
R40	Verbose Test	test_diary_poisoning_defence.py	d74e84f	sim	sim	não

3.3 Exemplos detalhados

01 – Condicional Test Logic (tests/test_local_files_path_arguments.py::test_string_paths_keep_file_operations)

Antes:

test_local_files_path_arguments.py

```python
@pytest.mark.parametrize('operation', ['list', 'read', 'write', 'append', 'delete'])
@pytest.mark.parametrize('relative', [False, True])
def test_string_paths_keep_file_operations(private_files, operation, relative):
    tool, context, home = private_files
    target = home / ('123' if relative else 'notes.txt')
    target.write_text('original')
    path = target.name if relative else str(target)
    result = tool.run({'operation': operation, 'path': path, 'content': 'changed'}, context)
    assert result.success, result.reply_text
    if operation == 'read': assert result.reply_text == 'original'
    elif operation == 'write': assert target.read_text() == 'changed'
    elif operation == 'append': assert target.read_text() == 'originalchanged'
    elif operation == 'delete': assert not target.exists()
    else: assert target.name in result.reply_text
```

Depois:

test_local_files_path_arguments.py

```python
@pytest.mark.parametrize('operation, outcome_holds', [
    pytest.param('list', lambda result, target: target.name in result.reply_text, id='list'),
    pytest.param('read', lambda result, target: result.reply_text == 'original', id='read'),
    pytest.param('write', lambda result, target: target.read_text() == 'changed', id='write'),
    pytest.param('append', lambda result, target: target.read_text() == 'originalchanged', id='append'),
    pytest.param('delete', lambda result, target: not target.exists(), id='delete'),
])
@pytest.mark.parametrize('filename, to_path', [
    pytest.param('notes.txt', str, id='absolute'),
    pytest.param('123', lambda target: target.name, id='relative'),
])
def test_string_paths_keep_file_operations(private_files, operation, outcome_holds, filename, to_path):
    tool, context, home = private_files
    target = home / filename
    target.write_text('original')
    result = tool.run({'operation': operation, 'path': to_path(target), 'content': 'changed'}, context)
    assert result.success, result.reply_text
    assert outcome_holds(result, target)

```

Por que era um test smell: a cadeia if/elif escolhia uma asserção diferente segundo a operação. O CATSPY classifica ramificações e laços dentro do teste como Conditional Test Logic, pois os caminhos podem alterar o que é efetivamente verificado. Aqui, cada operação devia ter um resultado esperado explícito.

Estratégia do CATSPY aplicada: separar os cenários condicionais para que cada caso tenha verificações próprias. O catálogo exemplifica a separação em métodos de teste. Neste projeto, a separação foi implementada por pytest.param: cada combinação de operação e caminho

torna-se um caso de teste independente, com a verificação fornecida como dado. A função não contém a cadeia condicional.

Métricas do teste refatorado:

Métrica	Antes	Depois
Complexidade ciclomática da função (cc_por_funcao)	13	3
SLOC do arquivo de teste (raw_por_arquivo_e_total)	37	41

02 – Sleepy Test (tests/test_chat_submission.py::test_cancel_drops_reply_and_emits_complete_none)

Antes:

test_chat_submission.py

```
def test_cancel_drops_reply_and_emits_complete_none(self, monkeypatch):
    """cancel_active_chat_query sets the per-query flag so the worker drops
    the reply (complete(None)) instead of displaying it."""
    _install_dialogue_memory(cfg=object(), db=object())
    started = threading.Event()

    def slow_engine(*a, **k):
        started.set()
        time.sleep(0.3)  # let cancel fire mid-run
        return "the reply that should be dropped"

    monkeypatch.setattr("jarvis.reply.engine.run_reply_engine", slow_engine)
    events = []
    daemon.submit_text_query(
        "hi", on_complete=lambda r: events.append(("complete", r)),
    )
    assert started.wait(timeout=2)
    daemon.cancel_active_chat_query()
    _wait_for_complete(events)
    assert events[-1] == ("complete", None)

```

Depois:



Por que era um test smell: time.sleep(0.3) tentava dar tempo para a resposta ser cancelada enquanto o motor ainda trabalhava. A duração fixa dependia da velocidade da máquina: podia desperdiçar tempo ou terminar antes do ponto necessário.

Estratégia do CATSPY aplicada: usar mecanismos de sincronização ou callbacks que dependam da conclusão da tarefa, em vez de uma pausa explícita. O teste usa threading.Event: aguarda o início do motor, cancela a consulta e só então liberta o motor. O timeout do wait serve para impedir bloqueio indefinido; não introduz uma espera fixa de 0,3 segundo.

Métricas do teste refatorado:

Métrica	Antes	Depois
Complexidade ciclomática da função (cc_por_funcao)	3	3
SLOC do arquivo de teste (raw_por_arquivo_e_total)	548	556

03 – Unknown Test (tests/test_graph_mutation_listener.py::test_unregister_is_idempotent)

Antes:

test_graph_mutation_listener.py

```
    def test_unregister_is_idempotent(self):
        def cb(**_):
            pass

        register_graph_mutation_listener(cb)
        unregister_graph_mutation_listener(cb)
        unregister_graph_mutation_listener(cb)  # second remove must not raise

```

Depois:

test_graph_mutation_listener.py

```
    def test_unregister_is_idempotent(self, graph_store):
        events = []

        def cb(**_):
            events.append(True)

        register_graph_mutation_listener(cb)
        unregister_graph_mutation_listener(cb)
        unregister_graph_mutation_listener(cb)
        graph_store.create_node("Bob", "a user fact", parent_id=BRANCH_USER)
        assert events == []

```

Por que era um test smell: o teste registava e removia um listener duas vezes, mas não continha asserção. Passar sem exceção verificava apenas parte do comportamento esperado; não confirmava que o listener deixara de receber eventos.

Estratégia do CATSPY aplicada: introduzir asserções adequadas para validar explicitamente o resultado da operação. O teste cria uma mutação depois da remoção e verifica que nenhum evento chegou ao listener.

Métricas do teste refatorado:

Métrica	Antes	Depois
Complexidade ciclomática da função (cc_por_funcao)	1	2
SLOC do arquivo de teste (raw_por_arquivo_e_total)	166	169

04 – Verbose Test
(tests/test_dialogue_memory.py::test_dialogue_memory_filters_tool_calls)

Antes:

test_dialogue_memory.py

```python
@patch('src.jarvis.reply.engine.chat_with_messages')
@patch('src.jarvis.reply.engine.extract_text_from_response')
@patch('src.jarvis.reply.engine.run_tool_with_retries')
def test_dialogue_memory_filters_tool_calls(self, mock_tool, mock_extract, mock_chat):
    """Test that JSON tool calls are filtered from dialogue memory."""
    # Mock dependencies
    mock_tool.return_value = Mock(reply_text="Weather data", error_message=None)

    # Mock multi-turn conversation: structured tool call then final response
    mock_chat.side_effect = [
        {
            "message": {
                "content": "",
                "tool_calls": [{
                    "id": "call_12345",
                    "function": {
                        "name": "webSearch",
                        "arguments": {"query": "London weather"}
                    }
                }]
            }
        },
        {"message": {"content": "It's sunny in London today!"}}
    ]
    mock_extract.side_effect = [
        "",  # Empty content for tool call
        "It's sunny in London today!"
    ]

    # Mock database and config
    mock_db = Mock()
    mock_cfg = Mock()
    mock_cfg.ollama_base_url = "http://localhost:11434"
    mock_cfg.ollama_chat_model = "test"
    mock_cfg.llm_chat_model = "test"
    mock_cfg.voice_debug = False
    mock_cfg.llm_tools_timeout_sec = 8.0
    mock_cfg.llm_embedding_timeout_sec = 10.0
    mock_cfg.llm_chat_timeout_sec = 45.0
    mock_cfg.memory_enrichment_max_results = 5
    mock_cfg.location_ip_address = None
    mock_cfg.location_auto_detect = False
    mock_cfg.agentic_max_turns = 8

    # Create dialogue memory
    dialogue_memory = DialogueMemory()

    # Run reply engine
    result = run_reply_engine(
        db=mock_db,
        cfg=mock_cfg,
        tts=None,
        text="What's the weather in London?",
        dialogue_memory=dialogue_memory
    )

    # Check that dialogue memory was updated
    chunks = dialogue_memory.get_pending_chunks()
    assert len(chunks) == 2  # User message and assistant response stored separately

    # Should include user input and final response
    assert "User: What's the weather in London?" in chunks
    assert "Assistant: It's sunny in London today!" in chunks

    # Should NOT include the tool call
    for chunk in chunks:
        assert 'call_12345' not in chunk
```

Depois:

test_dialogue_memory.py

```python
    @staticmethod
    def _reply_cfg():
        return Mock(
            ollama_base_url="http://localhost:11434",
            ollama_chat_model="test",
            llm_chat_model="test",
            voice_debug=False,
            llm_tools_timeout_sec=8.0,
            llm_embedding_timeout_sec=10.0,
            llm_chat_timeout_sec=45.0,
            memory_enrichment_max_results=5,
            location_ip_address=None,
            location_auto_detect=False,
            agentic_max_turns=8,
        )

    @patch('src.jarvis.reply.engine.chat_with_messages')
    @patch('src.jarvis.reply.engine.extract_text_from_response')
    @patch('src.jarvis.reply.engine.run_tool_with_retries')
    def test_dialogue_memory_filters_tool_calls(self, mock_tool, mock_extract, mock_chat):
        """Test that JSON tool calls are filtered from dialogue memory."""
        mock_tool.return_value = Mock(reply_text="Weather data", error_message=None)

        # Structured tool call, then the final response
        mock_chat.side_effect = [
            {
                "message": {
                    "content": "",
                    "tool_calls": [{
                        "id": "call_12345",
                        "function": {
                            "name": "webSearch",
                            "arguments": {"query": "London weather"}
                        }
                    }]
                }
            },
            {"message": {"content": "It's sunny in London today!"}}
        ]
        mock_extract.side_effect = ["", "It's sunny in London today!"]
        dialogue_memory = DialogueMemory()

        run_reply_engine(
            db=Mock(),
            cfg=self._reply_cfg(),
            tts=None,
            text="What's the weather in London?",
            dialogue_memory=dialogue_memory
        )

        # Only the user turn and the final reply are stored, never the tool call
        assert dialogue_memory.get_pending_chunks() == [
            "User: What's the weather in London?",
            "Assistant: It's sunny in London today!",
        ]

```

Por que era um test smell: a função tinha 67 linhas no código anterior. Misturava a preparação de configuração e mocks, a execução do motor e várias asserções sobre a memória. O CATSPY descreve Verbose Test como um método excessivamente longo, geralmente acima de 30 linhas, que dificulta a leitura e a manutenção.

Estratégia do CATSPY aplicada: dividir o método extenso em partes menores e mais focadas. A preparação da configuração foi extraída para _reply_cfg, e o teste concentra a verificação na lista completa de mensagens guardadas. O catálogo mostra a separação de responsabilidades em métodos menores; aqui, a parte extraída é um método auxiliar de preparação.

Métricas do teste refatorado:

Métrica	Antes	Depois
Complexidade ciclomática da função (cc_por_funcao)	6	2
SLOC do arquivo de teste (raw_por_arquivo_e_total)	487	484

4. Estado final e comparação

Métrica	Antes	Depois	Variação
Total de test smells (TEMPY)	1740	1672	-68
Testes aprovados	3402	3402	0
Testes com falha	8	8	0
Cobertura total (%)	70,52	70,52	0
SLOC dos testes	36.438	36.488	50
Complexidade ciclomática média das funções de teste	2,9589	2,9559	−0,0030
Índice de manutenibilidade médio dos arquivos de teste	46,6969	46,7066	+0,0096

Test smell	Antes	Depois	Variação
Conditional Test Logic	744	727	-17
Redundant Print	418	403	-15
Exception Handling	147	145	-2
Verbose Test	140	135	-5
Unknown Test	87	76	-11

Test smell	Antes	Depois	Variação
Sleepy Test	85	67	-18
Programming Paradigms Blend	79	79	0
Non-Functional Statement	22	22	0
Undefined Test	18	18	0
Total	1740	1672	7,5

5. Análise dos resultados

Quais métricas melhoraram e quais pioraram?

Tivemos muitas melhoras após a refatoração. Obtivemos uma diminuição na quantidade de test smells que estavam em 1740 e foram para 1672, além disso na maioria dos tipos de test smell identificados também obtivemos redução no número deles, apenas Non-Functional Statement, Programming Paradigms Blend e Undefined Test não sofreram mudanças. Em relação a execução dos testes não tivemos mudanças, os status de testes com sucesso, falha e ignorados continuaram os mesmos, assim como a cobertura dos testes. Obtivemos também uma leve diminuição da complexidade ciclomática, um aumento leve no índice de manutenibilidade e um leve aumento no SLOC dos testes.

Alguma refatoração introduziu novos test smells? Quais?

Não, nenhuma refatoração introduziu um novo test smell.

O resultado dos testes ou a cobertura mudou?

Não, os status dos testes continuam os mesmos, número de sucesso, falhas e ignorados, e a cobertura não teve variação.

Quais tipos de smell foram mais fáceis e mais difíceis de refatorar?

Os smells mais fáceis de refatorar foram o Unknown Test e o Verbose Test, pois, na maioria dos casos, bastou adicionar verificações com assert ou remover trechos desnecessários do código, deixando os testes mais objetivos e claros. Já os mais difíceis foram o Sleepy Test e o Conditional Test Logic. No Sleepy Test, a principal dificuldade foi substituir o time.sleep() por alternativas que não dependessem de esperas fixas, como o uso de eventos e simulação da passagem do tempo, garantindo que os testes continuassem funcionando corretamente. No Conditional Test Logic, foi necessário analisar as condições existentes e encontrar formas de simplificar a lógica sem comprometer as verificações realizadas pelos testes.

6. Uso do CATSPY

Como você usou o CATSPY durante a atividade?

Utilizamos o CATSPY como material de apoio para entender melhor os Test Smells identificados no projeto. Consultamos as definições, os exemplos e as sugestões de refatoração para compreender os problemas e pensar em como resolvê-los. Também usamos o catálogo para comparar as alterações realizadas com as boas práticas apresentadas.

Quais partes do catálogo mais ajudaram (definição, efeitos negativos, exemplo com problema, exemplo refatorado)?

As partes que mais nos ajudaram foram os exemplos com problema e os exemplos refatorados, pois facilitaram a visualização de como o código poderia ser melhorado. As definições também foram importantes para entender cada smell, enquanto os efeitos negativos ajudaram a compreender por que era necessário realizar as refatorações.

Houve casos em que a estratégia do catálogo não se aplicava diretamente ao projeto? Como você resolveu?

Sim, principalmente no Sleepy Test, pois alguns testes envolviam operações assíncronas e não bastava simplesmente remover o time.sleep(). Nesses casos, precisamos adaptar a solução utilizando threading.Event para sincronizar a execução e monkeypatch para simular a passagem do tempo. Dessa forma, conseguimos eliminar as esperas fixas sem comprometer o comportamento dos testes.

O que faltou ou poderia ser melhorado no catálogo?

Acredito que o catálogo poderia apresentar mais exemplos práticos de refatoração, principalmente envolvendo testes assíncronos e situações mais complexas, além de explicar melhor quando cada estratégia deve ser utilizada. Isso facilitaria a aplicação das soluções em projetos reais.

7. Divisão do trabalho

Integrante	Refatorações realizadas (IDs da Seção 3.2)	Quantidade
Henrique	[R01 a R08], [R12 a R16], [R25 a R29], [R36 a R37]	20
Tobias	[R09 a R11], [R17 a R24], [R30 a R35], [R38 a R40]	20

8. Dificuldades e lições aprendidas

Quais foram as principais dificuldades (ambiente, dependências, entendimento do código, ferramentas, revisão do PR)?

As principais dificuldades foram entender a estrutura do projeto e o funcionamento de alguns testes, já que o Jarvis possui uma base de código relativamente grande e utiliza várias bibliotecas. Também tivemos algumas dificuldades para compreender a lógica dos testes assíncronos e encontrar alternativas para remover os smells sem alterar o comportamento esperado. Em relação às ferramentas, precisei me familiarizar melhor com o pytest e seus recursos, como o monkeypatch. Já na organização das branches e dos commits, o principal cuidado foi manter cada refatoração separada para facilitar a análise das alterações.

O que você aprendeu com a atividade?

Aprendemos que não basta um teste funcionar, ele também precisa ser bem escrito, fácil de entender e confiável. A atividade nos ajudou a identificar diferentes Test Smells e compreender como eles podem prejudicar a qualidade dos testes. Também aprendemos novas formas de realizar verificações com assert, simular a passagem do tempo e controlar operações assíncronas sem depender de esperas fixas. Além disso, conseguimos praticar a leitura de código de um projeto open source e entender melhor a importância de realizar refatorações pequenas e organizadas, mantendo o comportamento original dos testes.

9. Contribuição (Pull Request)

Enviamos o PR para o repositório original por meio da branch refactor/all-test-smells-pr que contém todas as refatorações reunidas. Ainda não tivemos nenhuma resposta do mantenedor do repositório, estamos aguardando ele aprovar o rejeitar nosso PR.

10. Observações gerais

Durante a atividade, tivemos algumas dificuldades apenas na instalação das dependências do projeto, mas conseguimos resolvê-las. No restante, conseguimos utilizar bem as ferramentas e realizar as refatorações sem grandes problemas. Também optamos por organizar as alterações em commits separados, facilitando o acompanhamento e a revisão do código.